

Sprint 13 Full Structured Notes: Performance Optimization, Metrics, Tools, Web APIs, and CSS/JavaScript Optimization

0. Where You Are Before Sprint 13

Before Sprint 13, you should already understand these React ideas:

- React is a JavaScript library for building user interfaces.
- Vite helps you create and run React projects.
- JSX lets you write UI-like syntax inside JavaScript.
- Components are reusable pieces of UI.
- Props pass data from parent components to child components.
- `children` lets one component wrap other JSX.
- Conditional rendering lets React show different UI based on data.
- `map()` renders lists.
- Stable keys help React track list items.
- Events let React respond to clicks, typing, and form submissions.
- Controlled inputs store form values in React state.
- `useState` lets React remember changing data.
- State updates should be immutable.
- `useReducer` can organize related state logic.
- `useRef` can access DOM elements or store values that should not cause re-renders.
- `useEffect` runs side effects after React renders.
- Cleanup functions stop old timers, listeners, and subscriptions.
- Custom hooks let you reuse hook logic.
- Context API lets you share app-level values like user, theme, or language.
- Data fetching lets React communicate with APIs or backend services.
- Loading, data, and error states make async UI clear.
- `useMemo`, `useCallback`, and `React.memo` help reduce unnecessary React work in some cases.
- React Router connects URLs to React components.
- Lazy-loaded routes can avoid loading every page immediately.
- State management tools help organize shared state in larger apps.
- CSS frameworks and preprocessors help organize styling.
- Browser rendering turns HTML, CSS, JavaScript, images, and fonts into pixels on the screen.
- Real performance means measurable speed.
- Perceived performance means how fast the app feels to the user.
- INP measures how quickly the page responds visually after user interaction.
- Layout shift happens when visible content moves unexpectedly.

Sprint 12 taught you what performance means.

Sprint 13 teaches you how to measure performance, find real problems, and improve them.

Simple version:

Sprint 13 teaches you how to stop guessing why an app feels slow.

1. Sprint 13 Goal

The goal of Sprint 13 is to measure and improve React app performance.

By the end of Sprint 13, you should be able to:

- Explain why measuring comes before optimizing.
- Explain load-time optimization.
- Explain time to usable.
- Improve perceived performance with feedback and skeleton screens.
- Explain common performance metrics.
- Understand FCP, Speed Index, TBT, INP, Bounce Rate, and Unique Users.
- Use Chrome DevTools Network tab to inspect files and requests.
- Use Chrome DevTools Performance tab to inspect slow interactions.
- Use Lighthouse as a diagnostic tool.
- Understand PageSpeed Insights and WebPageTest at a beginner level.
- Explain what RUM tools measure.
- Use `performance.now()` to measure code duration.
- Understand the Performance Timing API.
- Understand `PerformanceObserver` at a beginner level.
- Optimize images.
- Understand caching.
- Understand compression.
- Reduce unnecessary CSS.
- Avoid expensive CSS selectors and animations.
- Reduce heavy JavaScript work.
- Use React performance tools carefully.
- Build a small Performance Dashboard project.

Simple version:

Sprint 13 teaches you how to make a React app faster by measuring first, fixing the real bottleneck, and checking the result.

2. The Most Important Sprint 13 Rule

Do not optimize by guessing.

Bad approach:

```
App feels slow
↓
Randomly add useMemo everywhere
↓
Randomly lazy-load everything
↓
Randomly compress some files
↓
Hope the app becomes faster
```

Better approach:

```
Measure first
↓
Find the bottleneck
↓
Make one improvement
↓
Measure again
↓
Keep or reject the change
```

Simple definition:

Optimization means making a measured problem better.

If you do not measure, you may fix the wrong thing.

Example:

You may think the React component is slow, but the real problem may be a huge image.

Or you may think the image is the problem, but the real problem may be a heavy JavaScript click handler.

Beginner rule:

Measure first. Optimize second.

3. Big Mental Model: Why a React App Can Feel Slow

A React app can be slow for many different reasons.

Slow to download?

→ File size, images, fonts, too many requests

Slow to show first content?

→ Render-blocking CSS/JS, bad source order, heavy startup work

Slow to become usable?

→ Too much JavaScript before interaction is possible

Slow after clicking?

→ Heavy event handlers, long tasks, unnecessary rendering

Feels slow even if technically fast?

→ No feedback, no skeleton, no loading state

Sprint 13 is about matching the problem to the correct fix.

Do not use the same fix for every performance problem.

Example:

- Large image problem → compress or resize image.
- Slow click problem → reduce JavaScript work in the click handler.
- Blank loading screen problem → add loading UI or skeleton screen.
- Huge first JavaScript bundle problem → code split or lazy-load routes.
- Too many repeated calculations problem → consider `useMemo` after measuring.

4. Load-Time Optimization

4.1 What Is Load Time?

Load time means how long it takes for the app's files to arrive and begin showing useful content.

Simple definition:

Load time is the time from opening the app to seeing useful content.

A React app usually needs several files:

- HTML
- CSS
- JavaScript
- Images
- Fonts

- Icons
- API data

If these files are too large or too many, the app can load slowly.

4.2 Common Load-Time Problems

Common problems include:

- Large images
- Large videos
- Too many network requests
- Large JavaScript bundles
- Large CSS files
- Unused CSS
- Unused JavaScript
- Fonts that are too large
- No caching strategy
- No compression
- Non-critical scripts blocking the page

Beginner rule:

The browser cannot show or run files it has not downloaded yet.

5. Large Images and Videos

Images and videos are often some of the largest files on a website.

A large image can slow down the page even if your React code is fine.

5.1 Bad Image Example

```

```

Problems:

- The file may be too large.
 - PNG may not be the best format for this image.
 - There is no `width` or `height`.
 - The browser may not reserve space for the image.
 - The image may cause layout shift when it loads.
 - The image is not lazy-loaded.
-

5.2 Better Image Example

```

```

Why this is better:

WebP → usually smaller file than PNG or JPEG
width/height → reserves image space and helps prevent layout shift
loading="lazy" → delays off-screen image loading
alt → keeps the image accessible

5.3 Image Optimization Rules

Use these beginner rules:

- Use the correct image size.
- Do not upload a 4000px image if you display it at 400px.
- Use modern formats such as WebP when suitable.
- Use AVIF/AV1-style formats when supported and appropriate.
- Compress images before shipping them.
- Add `width` and `height` to reserve layout space.
- Use `loading="lazy"` for images that are not needed immediately.
- Keep meaningful `alt` text for images that communicate content.
- Use empty `alt=""` only for decorative images.

Beginner rule:

A smaller image that still looks good is usually better than a huge image.

6. Too Many Requests

Every file can create a network request.

Examples:

```
logo.png
hero.png
background.png
button-icon.png
font-1.woff2
font-2.woff2
app.css
layout.css
cards.css
main.js
analytics.js
```

Each request takes work.

Too many requests can slow down loading, especially on weaker networks.

6.1 Better Request Strategy

Better approach:

```
Use only needed assets
Combine where reasonable
Inline critical CSS only when useful
Lazy-load non-critical parts
Cache files that rarely change
Remove unused scripts
Remove unused styles
```

Important warning:

Do not combine everything blindly.

Modern build tools already bundle and split files.

The real goal is not always “fewer files.”

The real goal is:

Send the right files at the right time.

6.2 One Giant File Can Also Be Bad

Bad idea:

```
Put the whole app into one huge JavaScript file
```

Problem:

The first page must download code for every feature, even features the user may never visit.

Better idea:

```
Load the main page first  
Load large pages or features only when needed
```

This is where route lazy loading and code splitting help.

7. Caching

7.1 What Is Caching?

Caching means the browser reuses files it already downloaded.

Simple definition:

Caching lets the browser avoid downloading the same file again.

Example:

```
First visit:  
download app.js, app.css, logo.webp  
  
Second visit:  
reuse cached app.js, app.css, logo.webp
```

Caching helps repeat visits feel faster.

7.2 What Should Be Cached?

Good files to cache:

- Logo files
- CSS files
- JavaScript build files
- Fonts
- Icons
- Images that do not change often

Be careful with files that change often.

7.3 Versioned File Names

A production build may create files like this:

```
app.9fd23ab.js  
style.a82bc91.css
```

The random-looking part is usually a hash.

If the file changes, the hash changes.

Example:

```
Old file:  
app.9fd23ab.js  
  
New file:  
app.41ac902.js
```

The browser sees the new name and downloads the new file.

Beginner rule:

Files that rarely change should be cached. Files that change should use versioned names.

8. Minification and Compression

8.1 What Is Minification?

Minification removes unnecessary characters from code.

Readable development JavaScript:

```
function calculateTotal(price, quantity) {  
  return price * quantity;  
}
```

Minified JavaScript idea:

```
function t(n,r){return n*r}
```

Minification can remove:

- Extra spaces
- New lines
- Comments
- Long variable names where safe

Simple definition:

Minification makes code smaller for the browser to download.

8.2 What Is Compression?

Compression reduces file size while sending files over the network.

Common examples:

- Gzip
- Brotli

Simple definition:

Compression packs files smaller before sending them to the browser.

Beginner mental model:

```
Original file
↓
Compressed on server
↓
Sent over network
↓
Browser decompresses it
↓
Browser uses the file
```

Beginner rule:

Minification makes code smaller. Compression sends it more efficiently.

9. Time to Usable

9.1 What Is Time to Usable?

A page can be visible but not usable.

Example:

```
User sees the dashboard
↓
User clicks a button
↓
Nothing happens because JavaScript is still busy
```

That is bad time to usable.

Simple definition:

Time to usable means how long it takes before the user can actually interact with the app smoothly.

9.2 Why Time to Usable Matters

Users do not only care that something appears.

They also care whether it works when they click, type, or scroll.

Bad experience:

Page appears

↓

User clicks

↓

App freezes

Better experience:

Page appears

↓

Important buttons work quickly

↓

Heavy features load later

10. Improving Time to Usable

10.1 Show Critical Content First

Better:

```
<main>
  <h1>Dashboard</h1>
  <p>Your latest project summary appears here.</p>

  <LargeChart />
</main>
```

Worse:

```
<main>
  <LargeChart />

  <h1>Dashboard</h1>
  <p>Your latest project summary appears here.</p>
</main>
```

Why the first version is better:

- The user sees the important message first.
- Heavy UI can load later.
- The page feels more useful sooner.

Beginner rule:

Put important content and actions early.

Heavy things that can often wait:

- Large charts
- Maps
- Carousels
- Animations
- Analytics scripts
- Non-critical widgets
- Large admin pages

10.2 Defer Non-Critical Scripts

Bad:

```
<script src="analytics.js"></script>
<h1>Welcome</h1>
```

This can block the browser before it shows the heading.

Better:

```
<h1>Welcome</h1>
<script src="analytics.js" defer></script>
```

`defer` tells the browser:

Download this script, but do not block parsing the page.

Use `defer` for scripts that are not needed immediately to show the first useful content.

Examples:

- Analytics
- Non-critical widgets
- Extra behavior that can wait

Beginner rule:

Do not block the first screen with scripts that are not needed immediately.

10.3 Lazy-Load Large Route Pages

Lazy loading means loading something only when it is needed.

Example:

```
import { lazy, Suspense } from "react";

const Dashboard = lazy(() => import("./pages/Dashboard"));

export default function App() {
  return (
    <Suspense fallback=<p>Loading dashboard...</p>>
      <Dashboard />
    </Suspense>
  );
}
```

What this means:

Do not load Dashboard immediately.
Load Dashboard when React needs to show it.

This is useful when Dashboard is large.

Beginner rule:

Do not make the first page download code for every possible page.

Important warning:

Do not lazy-load tiny components everywhere.

That can make code more complex without real benefit.

Good lazy-loading candidates:

- Dashboard pages
- Admin panels
- Large chart pages
- Settings pages
- Rarely visited routes
- Heavy modals

Poor lazy-loading candidates:

- Small buttons
 - Small labels
 - Tiny reusable components
 - Components needed immediately on the first screen
-

11. Perceived Performance

11.1 What Is Perceived Performance?

Perceived performance means how fast the app feels to the user.

Simple definition:

Perceived performance is the user's feeling of speed.

A page can technically take a few seconds but still feel acceptable if it gives useful feedback early.

A page can also load technically fast but feel bad if the user sees nothing or if the app freezes.

11.2 Bad Perceived Performance

Bad example:

```
<button>Save</button>
```

User clicks.

Nothing changes.

The user may think:

```
Did the click work?  
Is the app broken?  
Should I click again?
```

11.3 Better Perceived Performance

Better example:

```
<button disabled={isSaving}>
  {isSaving ? "Saving..." : "Save"}
</button>
```

Now the user gets immediate feedback.

This tells the user:

```
The click worked.
The app is saving.
Wait for the result.
```

Beginner rule:

Do not leave the user guessing. Show feedback during slow work.

12. Loading UI and Skeleton Screens

12.1 Normal Loading Message

This is acceptable for simple cases:

```
if (loading) {
  return <p>Loading...</p>;
}
```

It tells the user that something is happening.

But for larger pages, a skeleton screen can feel better.

12.2 What Is a Skeleton Screen?

A skeleton screen is a placeholder layout that looks like the shape of the content that is loading.

Simple definition:

A skeleton screen shows the page structure before the real data arrives.

Example:


```
if (loading) {  
  return (  
    <main>  
      <div className="skeleton title"></div>  
      <div className="skeleton line"></div>  
      <div className="skeleton line"></div>  
    </main>  
  );  
}
```

CSS:

```
.skeleton {  
  background: #e5e7eb;  
  border-radius: 8px;  
  margin-bottom: 12px;  
}  
  
.skeleton.title {  
  width: 50%;  
  height: 32px;  
}  
  
.skeleton.line {  
  width: 100%;  
  height: 16px;  
}
```

A skeleton tells the user:

```
The layout is coming.  
The app is working.  
You are not stuck.
```

12.3 When to Use Skeleton Screens

Use skeleton screens when:

- A page has a clear layout.
- Data may take noticeable time to load.
- You want the user to understand what is coming.
- A blank page would feel confusing.

A simple `Loading...` message is enough when:

- The wait is very short.
- The UI is simple.
- The layout is not important.

Beginner rule:

Loading UI improves perceived performance, but it should match the page.

13. Performance Metrics

Performance metrics are numbers that help you understand performance.

Simple definition:

A metric is a measurement.

Sprint 13 discusses these metrics:

- FCP
 - Speed Index
 - TBT
 - INP
 - Bounce Rate
 - Unique Users
-

14. FCP: First Contentful Paint

14.1 What Is FCP?

FCP means First Contentful Paint.

Simple definition:

FCP measures when the first real content appears on the screen.

Content can include:

- Text
- Image
- SVG
- Canvas content

Beginner question:

How long does the user wait before seeing anything useful?

14.2 Why FCP Matters

If FCP is slow, the user may see a blank screen for too long.

Bad experience:

User opens page
↓
Blank screen
↓
Still blank
↓
Content finally appears

Better experience:

User opens page
↓
Heading or basic content appears quickly
↓
More content loads after

Beginner rule:

Show useful content as early as possible.

15. Speed Index

15.1 What Is Speed Index?

Speed Index measures how quickly the visible page fills in.

Simple definition:

Speed Index measures how fast the page visually becomes complete.

A page may show a small logo quickly but load the real content slowly.

Speed Index helps detect that.

Beginner question:

Does the page visually become complete quickly?

15.2 FCP vs Speed Index

FCP asks:

When does the first content appear?

Speed Index asks:

How quickly does the visible page fill in?

Example:

FCP is good:
Logo appears quickly.

Speed Index is bad:
Main content takes a long time to appear.

Both metrics matter.

16. TBT: Total Blocking Time

16.1 What Is TBT?

TBT means Total Blocking Time.

Simple definition:

TBT measures how much time JavaScript blocks the main thread during loading.

The main thread handles important browser work:

- JavaScript
- Rendering
- Layout
- Painting
- Responding to user input

If JavaScript is too busy, the browser may not respond quickly.

Beginner question:

Is JavaScript keeping the browser too busy?

16.2 Why TBT Matters

Bad situation:

```
Page appears
↓
JavaScript is still doing heavy work
↓
User clicks
↓
Browser cannot respond immediately
```

This makes the app feel frozen.

Beginner rule:

Heavy JavaScript can make a visible page feel unusable.

17. INP: Interaction to Next Paint

17.1 What Is INP?

INP means Interaction to Next Paint.

Simple definition:

INP measures how long it takes after a user interaction before the page visually responds.

Example:

```
User clicks button
↓
JavaScript runs
↓
React updates state
↓
Browser paints the updated UI
```

INP focuses on the delay between the interaction and the next visual update.

Beginner question:

```
When the user clicks or types, does the app respond quickly?
```

17.2 Why INP Matters

Users care about response.

Bad interaction:

```
Click
↓
Delay
↓
Nothing seems to happen
↓
UI finally changes
```

Better interaction:

```
Click
↓
Button immediately changes to Saving...
↓
Result appears
```

Beginner rule:

Good interaction performance means the app responds quickly after user actions.

18. Bounce Rate

18.1 What Is Bounce Rate?

Bounce rate means how many users leave without continuing.

Simple definition:

Bounce rate measures how often users leave after viewing only one page or without meaningful engagement.

A slow page can increase bounce rate.

Beginner question:

Are users leaving because the page feels slow or unhelpful?

18.2 Bounce Rate Is Not Only a Technical Metric

Bounce rate can be affected by:

- Slow loading
- Poor design
- Confusing content
- Wrong audience
- Bad mobile experience
- Broken navigation
- Poor first impression

So do not assume performance is always the only reason.

Beginner rule:

Bounce rate can suggest a problem, but it does not always prove the cause.

19. Unique Users

19.1 What Are Unique Users?

Unique users means how many distinct users visited or used the app.

Simple definition:

Unique users measure how many different people used the app.

This helps you understand impact.

Example:

```
A 2-second delay affecting 5 users is one issue.  
A 2-second delay affecting 50,000 users is a bigger issue.
```

19.2 Why Unique Users Matter for Performance

Unique users help you prioritize.

Example:

```
Problem A:  
Slow page affects 20 users.  
  
Problem B:  
Slow page affects 30,000 users.
```

Problem B probably deserves more attention.

Beginner rule:

Performance problems matter more when they affect more users or important user flows.

20. Tools for Performance Testing

Sprint 13 introduces several tools:

- Chrome DevTools Network tab
- Chrome DevTools Performance tab

- Lighthouse
- PageSpeed Insights
- WebPageTest
- RUM tools

Simple rule:

Tools help you see what is slow instead of guessing.

21. Chrome DevTools: Network Tab

21.1 What Is the Network Tab?

The Network tab shows files and requests loaded by the page.

It helps you answer:

Which files load?
Which files are largest?
How many requests happen?
Are images too large?
Is JavaScript too large?
Which request is slow?

21.2 Beginner Network Tab Exercise

Steps:

1. Open your React app in the browser.
 2. Open Chrome DevTools.
 3. Go to the Network tab.
 4. Reload the page.
 5. Sort by Size.
 6. Identify the largest file.
 7. Ask whether that file is necessary on initial load.
-

21.3 What to Look For in the Network Tab

Look for:

- Huge images
- Huge JavaScript bundles

- Huge CSS files
- Slow API requests
- Too many font files
- Files that return errors
- Files that are loaded but not needed immediately
- Cache status
- Compression status

Beginner rule:

If a file is large and needed early, it can slow down initial loading.

22. Chrome DevTools: Performance Tab

22.1 What Is the Performance Tab?

The Performance tab records what the browser does while loading or interacting.

It helps you answer:

What happens when the page loads?
What happens when I click?
Is JavaScript taking too long?
Are there long tasks?
Is rendering delayed?
Is layout or paint expensive?

22.2 Beginner Performance Tab Exercise

Steps:

1. Open DevTools.
 2. Go to the Performance tab.
 3. Click Record.
 4. Interact with the app.
 5. Stop recording.
 6. Look for long tasks or delayed rendering.
-

22.3 What Is a Long Task?

A long task is a piece of work that keeps the main thread busy for too long.

Simple definition:

A long task is JavaScript or browser work that blocks responsiveness.

If the main thread is busy, the browser may not respond quickly to input.

Beginner rule:

Long tasks can make clicks and typing feel delayed.

23. Lighthouse

23.1 What Is Lighthouse?

Lighthouse is a tool that creates a report about a page.

It can check:

- Performance
- Accessibility
- Best practices
- SEO

Simple definition:

Lighthouse gives a beginner-friendly report with performance clues.

23.2 Do Not Chase the Score Blindly

Bad approach:

```
Lighthouse score is 82
↓
Try random fixes to reach 100
```

Better approach:

```
Read the report
↓
Find the important issue
↓
Fix what matters for users
```

↓

Measure again

Beginner rule:

Lighthouse finds clues. You decide which clues matter.

24. PageSpeed Insights and WebPageTest

24.1 PageSpeed Insights

PageSpeed Insights helps test a deployed page.

It can show performance information and suggestions.

Beginner idea:

PageSpeed Insights is useful when your app is online, not just running locally.

24.2 WebPageTest

WebPageTest gives deeper loading breakdowns.

It can help inspect:

- Request timing
- Waterfall charts
- Loading order
- Repeat view behavior
- Different locations or network conditions

Beginner idea:

WebPageTest is useful when you want a deeper look at how a page loads.

24.3 Tool Comparison

DevTools → good for local investigation
Lighthouse → good for quick report
PageSpeed Insights → good for deployed page field/lab signals
WebPageTest → good for deeper load breakdown

25. RUM Tools

25.1 What Is RUM?

RUM means Real User Monitoring.

Simple definition:

RUM measures performance from real users' devices, browsers, and networks.

This matters because your laptop may be fast, but real users may have:

- Slower phones
- Weak networks
- Older browsers
- Low battery
- Background apps running
- Different geographic locations

25.2 Lab Tools vs Real User Monitoring

Lab tools measure controlled tests.

RUM measures real user experience.

Simple comparison:

Lab tools tell you what might happen.
Real user monitoring tells you what is happening.

Both are useful.

Beginner rule:

Test locally, but remember that real users may experience different performance.

26. Web Performance APIs

Sprint 13 introduces browser APIs for measuring performance:

- `performance.now()`

- Performance Timing API
- PerformanceObserver

These APIs let JavaScript read performance information from the browser.

27. `performance.now()`

27.1 What Is `performance.now()`?

`performance.now()` gives a precise timestamp.

Simple definition:

`performance.now()` helps you measure how long code takes.

Example:

```
const start = performance.now();

let total = 0;
for (let i = 0; i < 10_000_000; i++) {
  total += i;
}

const end = performance.now();

console.log(`Work took ${end - start}ms`);
```

27.2 How to Read It

Mental model:

```
Start timer
Do work
End timer
Compare times
```

This line records the start:

```
const start = performance.now();
```

This line records the end:

```
const end = performance.now();
```

This calculates the duration:

```
end - start
```

27.3 When to Use `performance.now()`

Use it for:

- Measuring expensive functions
- Comparing two approaches
- Learning how long a block of code takes
- Debugging performance experiments

Do not use it as your only production monitoring strategy.

Beginner rule:

```
performance.now()
```

 is useful for small measurement experiments.

28. Performance Timing API

28.1 What Is the Performance Timing API?

The Performance Timing API gives information about page loading.

Simple definition:

The browser records timestamps during page loading, and your code can inspect those timestamps.

Basic example:

```
window.addEventListener("load", () => {  
  const timing = performance.timing;  
  
  const pageLoadTime = timing.loadEventEnd - timing.navigationStart;
```

```
console.log("Page load time:", pageLoadTime, "ms");
});
```

28.2 What This Code Means

```
window.addEventListener("load", () => {
```

Wait until the page fully loads.

```
const timing = performance.timing;
```

Get timing information from the browser.

```
const pageLoadTime = timing.loadEventEnd - timing.navigationStart;
```

Calculate total page load time.

```
console.log("Page load time:", pageLoadTime, "ms");
```

Print the result.

28.3 Beginner Note About Newer APIs

Some newer code uses:

```
performance.getEntriesByType("navigation")
```

At beginner level, understand the main concept first:

The browser can expose timing data about page loading.

29. PerformanceObserver

29.1 What Is PerformanceObserver ?

PerformanceObserver lets you watch performance entries.

Simple definition:

`PerformanceObserver` lets your code observe performance measurements as they happen.

Example:

```
const observer = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    console.log(entry.name, entry.duration);
  }
});

observer.observe({ entryTypes: ["measure"] });

performance.mark("start-work");

let total = 0;
for (let i = 0; i < 5_000_000; i++) {
  total += i;
}

performance.mark("end-work");

performance.measure("heavy-work", "start-work", "end-work");
```

29.2 How This Example Works

This creates the observer:

```
const observer = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    console.log(entry.name, entry.duration);
  }
});
```

This tells it to watch measurement entries:

```
observer.observe({ entryTypes: ["measure"] });
```

This creates a start mark:

```
performance.mark("start-work");
```

This creates an end mark:

```
performance.mark("end-work");
```

This measures between the marks:

```
performance.measure("heavy-work", "start-work", "end-work");
```

29.3 Mental Model

```
performance.mark() creates named timestamps.  
performance.measure() measures between timestamps.  
PerformanceObserver watches and reports the result.
```

`PerformanceObserver` is more advanced than `performance.now()`, but it is powerful because it can watch many measurements in a structured way.

Beginner rule:

Use `performance.now()` for simple timing. Learn `PerformanceObserver` when you need more structured performance measurement.

30. CSS Performance

CSS can affect performance when it is:

- Too large
- Too complex
- Full of unused styles
- Full of deep selectors
- Causing repeated layout work
- Causing expensive painting
- Animating layout-heavy properties
- Loading many fonts

Simple definition:

CSS performance means writing styles that the browser can load, match, calculate, and paint efficiently.

31. Remove Unused CSS

31.1 The Problem

Bad:

```
/* 5000 lines imported from a framework */
```

when your app only uses:

```
3 buttons  
1 card  
1 form
```

The browser may download and process CSS that your app does not need.

31.2 Better Approach

```
Use only what the app needs.  
Avoid huge styling systems without reason.  
Remove unused CSS when possible.  
Choose frameworks intentionally.
```

Beginner rule:

Do not import a large styling system if you only need a few small styles.

32. Avoid Very Deep CSS Selectors

32.1 Bad Selector Example

```
.app .page .content .section .card .header .title span {  
  color: red;  
}
```

Problems:

- Hard to read
- Hard to maintain
- Tightly tied to HTML structure
- Can be more expensive for the browser to match
- Easy to break when markup changes

32.2 Better Selector Example

```
.card-title {  
  color: red;  
}
```

This is clearer and easier to reuse.

Beginner rule:

Prefer clear class names over very deep selector chains.

33. Avoid Layout-Heavy Animations

33.1 Bad Animation Example

```
.box {  
  transition: width 300ms;  
}  
  
.box:hover {  
  width: 500px;  
}
```

Changing `width` can trigger layout.

Layout means the browser recalculates sizes and positions.

That can become expensive.

33.2 Better Animation Example

```
.box {  
  transition: transform 300ms;  
}  
  
.box:hover {  
  transform: scale(1.05);  
}
```

Changing `transform` is usually cheaper because it can often avoid recalculating layout.

33.3 Animation Rule

Prefer animating:

- `transform`
- `opacity`

Be careful animating:

- `width`
- `height`
- `top`
- `left`
- `right`
- `bottom`
- `margin`
- `padding`

Beginner rule:

Animate `transform` and `opacity` when possible.

34. Limit Font Variants

34.1 Font Problem

Bad:

```
Roboto regular  
Roboto medium  
Roboto semibold
```

```
Roboto bold
Roboto italic
Roboto bold italic
Roboto black
```

Each font file can add download cost.

Too many font variants can slow the page.

34.2 Better Font Strategy

Better:

```
Use only the weights/styles you actually need.
```

Example:

```
Regular
Bold
```

is often enough for a small app.

Also use `font-display` where appropriate so text does not stay invisible for too long.

Beginner rule:

Fonts are design assets, but they are also performance assets.

35. JavaScript Performance

JavaScript can block the browser because it runs on the main thread.

Simple definition:

JavaScript performance means avoiding unnecessary or heavy JavaScript work that delays loading, rendering, or interaction.

Common JavaScript performance problems:

- Huge bundles
- Unused code

- Heavy loops
 - Expensive calculations during render
 - Heavy click handlers
 - Too many event listeners
 - Rendering huge lists all at once
 - Repeated work during re-renders
 - Loading code for pages the user has not visited
-

36. Heavy Click Handler Problem

36.1 Bad Example

```
function SearchButton() {  
  function handleSearch() {  
    let total = 0;  
  
    for (let i = 0; i < 2_000_000_000; i++) {  
      total += i;  
    }  
  
    console.log(total);  
  }  
  
  return <button onClick={handleSearch}>Search</button>;  
}
```

Problem:

```
User clicks  
↓  
Huge loop runs  
↓  
Browser cannot paint quickly  
↓  
UI feels frozen
```

The browser cannot respond smoothly because JavaScript is doing too much work.

36.2 Better Beginner Version

```
import { useState } from "react";
```

```

export default function SearchButton() {
  const [status, setStatus] = useState("Idle");

  function handleSearch() {
    setStatus("Searching...");

    setTimeout(() => {
      let total = 0;

      for (let i = 0; i < 50_000_000; i++) {
        total += i;
      }

      console.log(total);
      setStatus("Done");
    }, 0);
  }

  return (
    <section>
      <button type="button" onClick={handleSearch}>
        Search
      </button>
      <p>{status}</p>
    </section>
  );
}

```

This is not perfect optimization, but it teaches an important idea:

Give feedback first.
Then do expensive work.

36.3 Important Warning

`setTimeout(..., 0)` does not magically make heavy work cheap.

It only delays the work so the browser may get a chance to update the UI first.

For serious heavy computation, you may later learn about Web Workers.

Beginner rule:

Do less heavy work on the main thread.

37. React-Specific Performance Reminders

Sprint 8 introduced these tools:

- `useMemo`
- `useCallback`
- `React.memo`

Sprint 13 reminds you to use them carefully.

They are useful, but not automatic solutions.

38. `useMemo` Reminder

38.1 Good Use of `useMemo`

```
import { useMemo, useState } from "react";

export default function ProductSearch({ products }) {
  const [query, setQuery] = useState("");

  const filteredProducts = useMemo(() => {
    return products.filter((product) =>
      product.name.toLowerCase().includes(query.toLowerCase())
    );
  }, [products, query]);

  return (
    <main>
      <input
        value={query}
        onChange={(event) => setQuery(event.target.value)}
        placeholder="Search products"
      />

      <ul>
        {filteredProducts.map((product) => (
          <li key={product.id}>{product.name}</li>
        ))}
      </ul>
    </main>
  );
}
```

This is reasonable because:

Filtering a large list can be expensive.
The result only needs recalculation when products or query changes.

38.2 Bad Use of `useMemo`

```
const fullName = useMemo(() => firstName + " " + lastName, [firstName, lastName]);
```

Usually unnecessary.

Simple string joining is cheap.

Better:

```
const fullName = firstName + " " + lastName;
```

Beginner rule:

Memoization is for real repeated work, not for decorating every value.

39. `useCallback` Reminder

`useCallback` remembers a function reference.

It is useful when:

- You pass a function to a memoized child component.
- The child uses `React.memo`.
- The function prop would otherwise change on every render.
- You measured or noticed unnecessary child re-renders.

Do not use `useCallback` everywhere.

Beginner rule:

Use `useCallback` when function identity matters.

40. `React.memo` Reminder

`React.memo` can skip child re-renders when props stay the same.

It is useful when:

- The child component is expensive to render.
- The parent re-renders often.
- The child receives props that often stay the same.
- You measured unnecessary child rendering.

Do not wrap every component in `React.memo`.

Beginner rule:

`React.memo` is useful for expensive child components, not every component.

41. Rendering Huge Lists

A common performance problem is rendering too many items at once.

Bad example:

```
<ul>
  {products.map((product) => (
    <li key={product.id}>{product.name}</li>
  ))}
</ul>
```

This may be fine for 20 products.

It may be bad for 20,000 products.

Beginner improvement:

```
<ul>
  {products.slice(0, 20).map((product) => (
    <li key={product.id}>{product.name}</li>
  ))}
</ul>
```

This only renders the first 20.

For larger real apps, you may later learn list virtualization.

Beginner rule:

Do not render thousands of visible elements if the user only needs a few at a time.

42. Sprint 13 Mini Project: Performance Dashboard

Build a small React page that lets you practice measurement and optimization.

This project includes:

- A large product list
 - Search filtering
 - `useMemo`
 - `performance.now()`
 - A status message for perceived performance
 - An optimized image example
 - CSS for a clean layout
-

43. Mini Project File: `src/App.jsx`

```
import { useMemo, useState } from "react";
import "./App.css";

const products = Array.from({ length: 5000 }, (_, index) => ({
  id: index + 1,
  name: `Product ${index + 1}`,
  price: Math.floor(Math.random() * 100) + 1,
}));

export default function App() {
  const [query, setQuery] = useState("");
  const [status, setStatus] = useState("No measurement yet.");
  const [duration, setDuration] = useState(null);

  const filteredProducts = useMemo(() => {
    return products.filter((product) =>
      product.name.toLowerCase().includes(query.toLowerCase())
    );
  }, [query]);

  function handleMeasureWork() {
```

```

setStatus("Measuring heavy work...");

setTimeout(() => {
  const start = performance.now();

  let total = 0;
  for (let i = 0; i < 50_000_000; i++) {
    total += i;
  }

  const end = performance.now();
  setDuration(end - start);
  setStatus("Measurement complete.");
}, 0);
}

return (
  <main className="page">
    <section className="hero">
      <h1>Performance Dashboard</h1>
      <p>
        Sprint 13 teaches measurement first, then optimization. Do not guess.
      </p>
    </section>

    <section className="card">
      <h2>Search Products</h2>

      <label htmlFor="search">Search</label>
      <input
        id="search"
        value={query}
        onChange={(event) => setQuery(event.target.value)}
        placeholder="Try Product 100"
      />

      <p>Showing {filteredProducts.length} products.</p>

      <ul>
        {filteredProducts.slice(0, 20).map((product) => (
          <li key={product.id}>
            {product.name} - ${product.price}
          </li>
        ))}
      </ul>
    </section>

    <section className="card">

```

```

    <h2>Measure Heavy Work</h2>

    <button type="button" onClick={handleMeasureWork}>
      Measure work
    </button>

    <p>{status}</p>

    {duration !== null && (
      <p>Heavy work took {duration.toFixed(2)}ms.</p>
    )}
  </section>

  <section className="card">
    <h2>Optimized Image Example</h2>

    
  </section>
</main>
);
}

```

44. Mini Project File: `src/App.css`

```

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  background: #f8fafc;
  color: #111827;
}

.page {
  width: min(900px, 100%);
  margin: 0 auto;
}

```

```

    padding: 24px;
  }

.hero,
.card {
  background: white;
  border: 1px solid #e5e7eb;
  border-radius: 16px;
  padding: 20px;
  margin-bottom: 20px;
}

input {
  display: block;
  width: 100%;
  padding: 10px;
  margin: 8px 0 12px;
}

button {
  padding: 10px 14px;
  cursor: pointer;
}

img {
  max-width: 100%;
  height: auto;
  display: block;
}

```

45. What the Mini Project Teaches

useMemo → avoids repeated filtering work when unrelated state changes
 performance.now() → measures expensive code
 loading/status text → improves perceived performance
 width/height image attributes → reduce layout shift
 limited rendered list → avoids rendering thousands of items at once

45.1 `useMemo` in the Project

```

const filteredProducts = useMemo(() => {
  return products.filter((product) =>
    product.name.toLowerCase().includes(query.toLowerCase())
  )
}

```

```
);  
}, [query]));
```

This tells React:

Only recalculate `filteredProducts` when `query` changes.

45.2 `performance.now()` in the Project

```
const start = performance.now();  
// heavy work  
const end = performance.now();  
setDuration(end - start);
```

This measures how long the heavy work took.

45.3 Status Feedback in the Project

```
<p>{status}</p>
```

This tells the user what is happening.

45.4 Optimized Image in the Project

```

```

This practices image optimization habits.

46. Debug Task

Fix this bad code:

```
<script src="large-analytics.js"></script>
```



```


<button onclick="runVeryHeavyWork()">Search</button>
```

47. Debug Task Problems

Problems:

```
Script may block important content.
Image has no width/height, so it can cause layout shift.
Image may be too large or poorly optimized.
Inline onclick is not React style.
Heavy work may block interaction.
No user feedback.
```

48. Better HTML Idea

```
<h1>Search Page</h1>



<script src="large-analytics.js" defer></script>
```

Why this is better:

- Important content appears before the script.
 - `defer` prevents the script from blocking HTML parsing.
 - WebP may reduce image size.
 - `width` and `height` reserve space.
 - `loading="lazy"` delays off-screen loading.
 - `alt` keeps the image accessible.
-

49. Better React Button Idea

```
function SearchAction() {
  const [status, setStatus] = useState("Ready");

  function handleSearch() {
    setStatus("Searching...");

    setTimeout(() => {
      // Do reduced or delayed work here.
      setStatus("Done");
    }, 0);
  }

  return (
    <>
      <button type="button" onClick={handleSearch}>
        Search
      </button>
      <p>{status}</p>
    </>
  );
}
```

Why this is better:

- Uses React `onClick` instead of HTML `onclick`.
- Uses a named event handler.
- Uses `type="button"`.
- Shows status feedback.
- Delays heavy work so the UI can show feedback first.

50. Common Sprint 13 Beginner Mistakes

Mistake 1: Optimizing Without Measuring

Bad:

The app feels slow, so I added `useMemo` everywhere.

Better:

The app feels slow, so I checked DevTools first.

Mistake 2: Thinking Performance Is Only React

React performance matters, but performance also includes:

- Network
- Images
- CSS
- Fonts
- JavaScript
- Browser rendering
- User feedback
- Server response
- Caching
- Compression

Mistake 3: Lazy-Loading Everything

Lazy loading is useful for large code or assets.

It is not useful for every tiny component.

Bad:

Lazy-load every button, label, and card.

Better:

Lazy-load large pages or rarely used features.

Mistake 4: Ignoring Perceived Performance

Bad:

The app is doing work, but the user sees nothing.

Better:

Show Loading..., Saving..., skeleton screens, disabled buttons, or progress messages.

Mistake 5: Rendering Too Much at Once

Bad:

Render 10,000 list items when only 20 are visible.

Better:

Render only what the user needs now.

Mistake 6: Using Huge Images

Bad:

Use a 5MB image for a small card thumbnail.

Better:

Resize, compress, and use a modern format.

Mistake 7: Animating Layout Properties

Bad:

```
transition: width 300ms;
```

Better:

```
transition: transform 300ms;
```

51. Sprint 13 Practice Tasks

Warm-Up Drills

Do these first:

1. Write one sentence explaining why measuring comes before optimizing.
 2. Write the meaning of FCP in simple words.
 3. Write the meaning of INP in simple words.
 4. List three things that can make a page slow to load.
 5. List three things that can make a click feel slow.
-

Guided Coding Task

Build the Performance Dashboard mini project.

Requirements:

- Create a list of 5000 products.
 - Add a search input.
 - Filter the products.
 - Use `useMemo` for filtering.
 - Render only the first 20 results.
 - Add a button that measures heavy work with `performance.now()`.
 - Show a status message while measuring.
 - Add an optimized image example.
-

Debug Practice

Fix this code:

```
<script src="large-analytics.js"></script>

<button onclick="runVeryHeavyWork()">Search</button>
```

Explain every fix.

Tool Practice

Use Chrome DevTools:

1. Open the Network tab.

2. Reload your app.
3. Find the largest file.
4. Check whether the file is needed immediately.
5. Open the Performance tab.
6. Record a button click.
7. Look for long tasks.
8. Run Lighthouse.
9. Write down one useful recommendation.

Challenge Task

Improve the Performance Dashboard:

- Add a loading skeleton.
- Add a fake slow fetch with `setTimeout`.
- Show the skeleton while loading.
- Add an image with and without `width / height` and observe layout behavior.
- Compare filtering with and without `useMemo`.
- Use `performance.now()` to compare two versions of a function.

52. Revision Checkpoint Questions

Answer these without looking at notes:

1. Why should you measure before optimizing?
2. What is load-time optimization?
3. What is time to usable?
4. What is perceived performance?
5. Why can a page feel slow even if it technically loads quickly?
6. What is FCP?
7. What is Speed Index?
8. What is TBT?
9. What is INP?
10. What does Bounce Rate tell you?
11. Why do Unique Users matter when prioritizing performance issues?
12. What does the DevTools Network tab show?
13. What does the DevTools Performance tab show?
14. Why should you not chase a Lighthouse score blindly?
15. What does `performance.now()` measure?
16. What does the Performance Timing API expose?
17. What does `PerformanceObserver` do?
18. Why should images have `width` and `height`?
19. Why is `loading="lazy"` useful?
20. Why can too much JavaScript delay interaction?

21. Why are `transform` and `opacity` usually better animation targets?
 22. When is `useMemo` useful?
 23. When is `useMemo` unnecessary?
 24. Why should you not render thousands of list items at once?
 25. What should you do after making a performance improvement?
-

53. Definition of Done for Sprint 13

You are ready to move past Sprint 13 when you can do these without notes:

```
Explain why measuring comes before optimizing.
Explain FCP, Speed Index, TBT, and INP in beginner words.
Use DevTools Network tab to find large files.
Use DevTools Performance tab to inspect a slow interaction.
Use Lighthouse as a diagnostic tool, not just a score.
Use performance.now() to measure code duration.
Explain what PerformanceObserver is for.
Optimize an image with format, dimensions, alt text, and lazy loading.
Explain why caching helps repeat visits.
Explain why compression helps downloads.
Explain why too much JavaScript can delay interaction.
Explain why skeleton screens and instant feedback improve perceived performance.
Remove or reduce unnecessary CSS/JS.
Avoid layout-heavy animations where possible.
Build and explain the Performance Dashboard mini project.
```

54. Shortest Correct Mental Model

Sprint 13 teaches this:

A React app should not just work. It should be measured, optimized, and verified.

The correct process is:

```
Measure
↓
Find the real bottleneck
↓
Fix the bottleneck
```

↓

Measure again

Good performance means:

Users see useful content quickly.
Users get fast feedback when they interact.
The app avoids unnecessary downloads.
The app avoids unnecessary JavaScript work.
The app avoids unnecessary CSS/rendering work.
The app works well on real devices and networks.

Final beginner rule:

Do not guess. Measure first, improve carefully, then verify the result.